



Assignment 02

Submitted By

Ijlal Ahmad

Submitted to

Mr. Aqib Adeel

Roll No

FL-21565

Date

27 August 2024

Bubble Sort

Code

```
#include <iostream>

using namespace std;

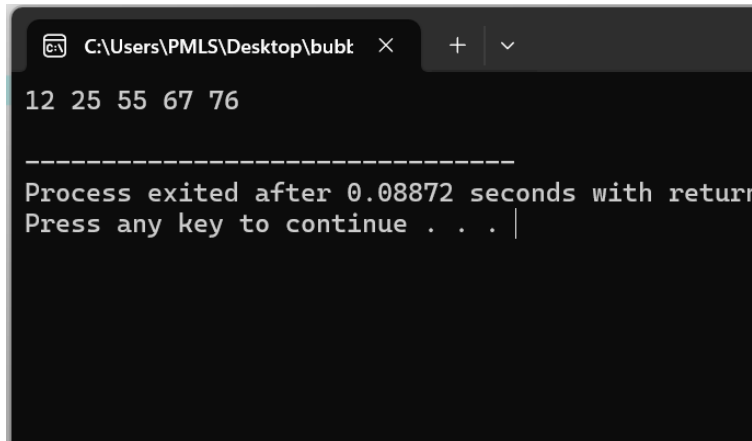
int main(){
    int arr[5]={67, 76, 25, 12, 55};

    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }

    for (int i = 0; i < 5; i++) {
        cout << arr[i] << " ";
    }

    cout << endl;
}
```

Output



```
C:\Users\PMLS\Desktop\bubt  X  +  v
12 25 55 67 76
-----
Process exited after 0.08872 seconds with return
Press any key to continue . . . |
```

Explanation:

The outer loop (for (int i = 0; i < 5; i++)) runs 5 times (equal to the length of the array).

The inner loop (for (int j = 0; j < 5-i-1; j++)) runs fewer times on each pass, because the largest elements "bubble up" to the end of the array in each iteration, and thus do not need to be checked again.

Inside the inner loop, there is a comparison between adjacent elements (if (arr[j] > arr[j+1])):

If the current element is greater than the next element, they are swapped using a temporary variable temp.

Working of the Code:

Initial Array: {67, 76, 25, 12, 55}

Pass 1:

The largest number 76 is moved to the end of the array.

Comparisons:

67 > 76 (no swap)

76 > 25 (swap, array becomes {67, 25, 76, 12, 55})

76 > 12 (swap, array becomes {67, 25, 12, 76, 55})

76 > 55 (swap, array becomes {67, 25, 12, 55, 76})

Pass 2:

The second largest number 67 is moved to the second last position.

Comparisons:

67 > 25 (swap, array becomes {25, 67, 12, 55, 76})

67 > 12 (swap, array becomes {25, 12, 67, 55, 76})

67 > 55 (swap, array becomes {25, 12, 55, 67, 76})

Pass 3:

The third largest number 55 is moved to the third last position.

Comparisons:

25 > 12 (swap, array becomes {12, 25, 55, 67, 76})

25 > 55 (no swap)

Pass 4:

The fourth largest number 25 is already in its correct position.

Comparisons:

12 > 25 (no swap)

Pass 5: The array is fully sorted.

Final Sorted Array: {12, 25, 55, 67, 76}

Selection Sort

Code

```
#include <iostream>

using namespace std;

int main(){

int arr[5]={64, 25, 12, 22, 11};

for (int i = 0; i < 5; i++) {

    int minIndex = i;

    for (int j = i+1; j < 5; j++) {

        if (arr[j] < arr[minIndex]) {

            minIndex = j;

        }

    }

    int temp = arr[minIndex];

    arr[minIndex] = arr[i];

    arr[i] = temp;

}


for (int i = 0; i < 5; i++) {

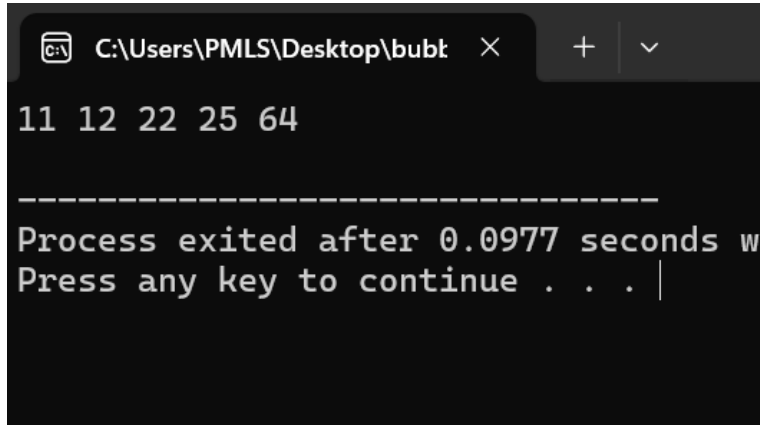
    cout << arr[i] << " ";

}

cout << endl;

}
```

Output

A screenshot of a C++ program's output in a terminal window. The window has a tab labeled 'C:\Users\PMLS\Desktop\bubt'. The output shows the numbers '11 12 22 25 64' on the first line, followed by a dashed line '-----' on the second line. The third line says 'Process exited after 0.0977 seconds with return code 0'. The fourth line says 'Press any key to continue . . . |' with a cursor at the end.

```
C:\Users\PMLS\Desktop\bubt X + v
11 12 22 25 64
-----
Process exited after 0.0977 seconds with return code 0
Press any key to continue . . . |
```

Explanation:

Outer Loop: `for (int i = 0; i < 5; i++)`

This loop iterates through each element of the array, treating `arr[i]` as the current position to be filled with the smallest element from the unsorted portion of the array.

Finding the Minimum: `int minIndex = i;`

The `minIndex` variable stores the index of the smallest element found in the unsorted portion of the array.

Inner Loop: `for (int j = i + 1; j < 5; j++)`

This loop searches through the unsorted portion of the array (from index `i+1` to the end) to find the smallest element.

If `arr[j] < arr[minIndex]`, `minIndex` is updated to `j`, which means the smallest element is now at index `j`.

After finding the smallest element in the unsorted portion, it is swapped with the element at the current position `i`. This places the smallest element in its correct position in the sorted portion of the array.

After sorting, the array is printed using a loop that iterates through each element of the array.

`cout << arr[i] << " ";` prints each element followed by a space.

`cout << endl;` moves the cursor to the next line after printing the array.

Working of the Code

Initial Array: {64, 25, 12, 22, 11}

Pass 1:

Find the smallest element and move it to the first position.

minIndex starts at 0 (value 64).

Compare with 25 → minIndex stays 0.

Compare with 12 → minIndex becomes 2.

Compare with 22 → minIndex stays 2.

Compare with 11 → minIndex becomes 4.

Swap 64 and 11 → Array becomes {11, 25, 12, 22, 64}.

Pass 2:

Find the next smallest element and move it to the second position.

minIndex starts at 1 (value 25).

Compare with 12 → minIndex becomes 2.

Compare with 22 → minIndex stays 2.

Compare with 64 → minIndex stays 2.

Swap 25 and 12 → Array becomes {11, 12, 25, 22, 64}.

Pass 3:

Find the next smallest element and move it to the third position.

minIndex starts at 2 (value 25).

Compare with 22 → minIndex becomes 3.

Compare with 64 → minIndex stays 3.

Swap 25 and 22 → Array becomes {11, 12, 22, 25, 64}.

Pass 4:

The remaining two elements are already in order.

minIndex stays at 3 (value 25), no swap needed.

Final Sorted Array: {11, 12, 22, 25, 64}